Logiques pour l’IA

Chapitre 1 — Logique classique et raisonnement symbolique

1.1. Propositions et connecteurs logiques

- Proposition, prémisse, conclusion et logique bivalente.
- Connecteurs NON, ET, OU, implication et équivalence.
- Tables de vérité et interprétation d’une formule.
- Tautologie, contradiction et équivalence logique.
- Lois de De Morgan et transformations simples des formules.

1.2. Raisonnements et règles d’inférence

- Validité d’un raisonnement et conséquence logique.
- Déduction, induction et abduction : principes et différences.
- Modus ponens et modus tollens.
- Syllogisme catégorique et syllogisme hypothétique.
- Raisonnement par enchaînement de règles et contre-exemple.

1.3. Satisfiabilité et cohérence

- Satisfiabilité et insatisfiabilité d’une formule.
- Modèle logique : affectation satisfaisant une formule.
- Cohérence d’un ensemble de prémisses ou d’une base de connaissances.
- Vérification d’une conclusion par recherche de contre-exemple.
- Notion de forme normale conjonctive (FNC), à titre de repère.

1.4. Bases de connaissances et systèmes experts

- Faits, règles SI…ALORS et base de connaissances.
- Architecture d’un système expert : base de connaissances et moteur d’inférence.
- Chaînage avant : application des règles à partir des faits.
- Chaînage arrière : recherche des prémisses nécessaires à un objectif.
- Fermeture déductive et arrêt lorsqu’aucun nouveau fait n’est produit.
- Intérêts et limites des systèmes experts.

1.5. Manipulation logique avec SymPy

- symbols : déclarer des propositions symboliques.
- And, Or, Not, Implies, Equivalent : construire des formules.
- subs : substituer des valeurs de vérité.
- simplify_logic et to_cnf : simplifier et transformer des formules.
- satisfiable : rechercher un modèle ou détecter une contradiction.
- Vérifier une inférence avec les prémisses et la négation de la conclusion.

Chapitre 2 — Logique floue et systèmes d’inférence

2.1. Fondements de la logique floue

- Différence entre logique classique et logique floue.
- Ensemble classique, ensemble flou et degré d’appartenance.
- Différence entre degré d’appartenance et probabilité.
- Univers de discours, variable linguistique et termes linguistiques.
- Rôle du contexte dans la définition des catégories floues.

2.2. Fonctions d’appartenance

- Fonctions triangulaire et trapézoïdale : formes et paramètres.
- Fonctions à épaulement gauche et droit.
- Fonction gaussienne : principe général.
- Chevauchement des ensembles flous et interprétation des degrés.
- Choix des bornes et des fonctions selon le problème étudié.

2.3. Opérateurs de logique floue

- Intersection (ET), union (OU) et complément (NON).
- ET flou : minimum et produit (t-normes).
- OU flou : maximum et somme probabiliste (t-conormes).
- Effet du choix des opérateurs sur les degrés obtenus.

2.4. Inférence floue et défuzzification

- Architecture d’un système d’inférence floue.
- Fuzzification des entrées.
- Règles SI…ALORS : antécédent, conséquent et degré d’activation.
- Méthode de Mamdani : implication par troncature et agrégation des conclusions.
- Défuzzification : objectif et méthode du centre de gravité (centroid).
- Autres méthodes : bisector, MOM, SOM et LOM (aperçu).
- Différence générale entre Mamdani et Sugeno.
- Influence de la couverture des règles et des paramètres sur la sortie.

2.5. Implémentation avec scikit-fuzzy

- trimf, trapmf, gaussmf : définir les fonctions d’appartenance.
- interp_membership : déterminer les degrés d’appartenance.
- numpy.minimum et numpy.maximum : opérations et agrégation.
- defuzz : calculer une sortie numérique.
- control.Antecedent et control.Consequent : déclarer les variables floues.
- control.Rule, ControlSystem, ControlSystemSimulation : construire et exécuter un moteur d’inférence.
- input, compute, output et view : renseigner, calculer, récupérer et visualiser les résultats.